

Conclusion

Summary I

This paper has presented `template_pools_rules_tools`, a meta-project exemplar demonstrating how research software projects embedded in a monorepo can integrate three categories of shared resources — data pools (fonds), governance rules, and executable tools — without tight coupling to any specific resource instance. The key contributions are:

1. **A four-module architecture** (`fonds_reader`, `rules_applier`, `tools_invoker`, `integration`) that provides a canonical pattern for resource-aware project code in the template repository. Each module is independently testable and independently deployable.
2. **A typed manifest convention** (`fonds.yaml`, `rules.yaml`, `tools.yaml`) that makes resource capabilities explicit and checkable at pipeline initialisation time, shifting failure detection from runtime to startup — a significant improvement for reproducibility [[@Wilson2014best](#)].

Summary II

3. **A token injection pipeline** that links manuscript prose to integration runtime statistics through `{{integration.*}}` tokens, ensuring that quantitative claims in the manuscript are always generated by the pipeline rather than authored manually. In the current run this covered `{{integration.fonds_loaded}}` fonds, `{{integration.rules_sets_ok}}` rule sets, `{{integration.tools_discovered}}` tools, and `{{integration.bib_entries}}` bibliography entries.
4. **A three-level resilience design** — resource absence, schema malformation, and script absence — that allows the pipeline to degrade gracefully and report failures informatively rather than crashing, consistent with best practices for robust research software [Taschuk2017ten].

Design Decisions Revisited I

Several design decisions deserve emphasis as lessons for future exemplar authors:

Repo-root-relative discovery is non-negotiable in a forkable monorepo. Any hard-coded absolute path or working-directory assumption will fail when the repository is cloned to a different location. The `pathlib.Path(__file__).resolve().parents[N]` idiom used throughout `src/` ensures that discovery works from any working directory.

Graceful fallbacks over exceptions is the correct trade-off for a resource-consumption layer. The alternative — raising `FileNotFoundError` when a fond is missing — would make the integration pipeline fragile to the ordering of parallel agent writes. Returning `None` and logging a warning allows the pipeline to produce a complete, if partial, result dict that downstream consumers can reason about.

Design Decisions Revisited II

Real-file tests over mocks is required for a template exemplar. Mocks can pass tests while the real discovery logic is silently broken. Tests that use real file paths exercise the full path-resolution chain and catch regressions that mocks would miss.

Thin orchestration scripts keep the scripts directory readable and focus all testable logic in `src/`. A script that does more than parse arguments, call a `src/` function, and write output is accumulating business logic that belongs elsewhere.

Future Directions I

The three-layer architecture described here is deliberately extensible. The most natural extension is a fourth resource category: **models** (`models/<scope>/<name>/`) for pre-trained machine learning models and their inference scripts. The pattern would follow exactly the same manifest-and-reader design, with `models.yaml` declaring type, version, and entrypoints, and a `models_loader` module providing `discover_models()` and `validate_model_files_exist()`. Adding this layer to `template_pools_rules_tools` would require only a new reader module and a new section in the integration orchestrator — the existing architecture places no constraints on the number of resource categories.

Future Directions II

A second direction is **cross-fond validation**: checking that cite keys used in a project's manuscript/references.bib are a subset of those available in `fonds/templates/template_bibliography/`. This would require a small cross-reference function in the integration module and would strengthen the reproducibility guarantee by ensuring that manuscript citations always trace back to curated, shared sources.

The exemplar is ready to fork. A project team wishing to adopt this architecture need only copy the four `src/` modules, update the resource directory paths in each reader, and replace the exemplar `fond/rules/tool` names with their own.